

ABSTRACT

Scheduling and event management are important for personal productivity, academic activities, and organizational coordination. Many existing calendar applications are either limited in functionality, difficult to customize, or closely connected to specific platforms. To overcome these limitations, the project **“Event Management Application With Scheduling and Tracking”** was developed as a modern web-based scheduling and event management system with an interactive user interface and secure backend support.

The main objective of this project is to provide users with an efficient and user-friendly platform for managing events, schedules, and tasks. The application supports multiple calendar views such as day, week, month, year, and agenda views, allowing users to organize and track events easily. Users can create, edit, delete, drag, and resize events directly within the calendar interface, improving usability and scheduling flexibility.

The frontend of the application is developed using Next.js, React, and TypeScript to ensure scalability and maintainability. Tailwind CSS and ShadCN UI components are used to create a responsive and visually appealing interface. The system also supports dark mode, smooth UI transitions, and persistent user preferences for an enhanced user experience.

The backend is implemented using Node.js and Express.js, while MongoDB is used as the database for storing user and event information. Authentication and authorization are handled using JSON Web Tokens (JWT), and passwords are securely protected through hashing techniques. RESTful API endpoints are used for user authentication and event-related operations such as event creation, updating, deletion, and retrieval.

This project demonstrates the integration of frontend and backend technologies into a complete full-stack web application. It also highlights concepts such as modular architecture, API integration, responsive design, authentication, database management, and state management.

The “**Event Management Application With Scheduling and Tracking**” can be used for personal scheduling, educational planning, and small team coordination. Future enhancements may include real-time collaboration, smart notifications, AI-based scheduling assistance, and integration with external calendar services. Overall, the project provides a scalable, secure, and modern solution for event scheduling and tracking while also serving as a practical learning platform for full-stack application development.

CHAPTER 1

INTRODUCTION

1.1 COMPANY PROFILE

1.2 PROJECT OVERVIEW

The objective of this project is the development of a high-performance scheduling engine built on the **Full Calendar** framework. Rather than a static calendar, this application functions as a dynamic event management suite, offering granular control through daily, weekly, and monthly temporal views.

The core philosophy of the build focuses on **reducing friction** in time management. By implementing native drag-and-drop APIs and real-time state updates, the application allows users to manipulate complex schedules with minimal cognitive load. The frontend is structured using a modular architecture, ensuring that the interface remains performant even as event density increases. Key features include:

- **Static Layouts:** Many systems struggle to transition between micro-level (daily) and macro-level (monthly) planning without losing context.
- **Interaction Lag:** A lack of intuitive UI gestures (like drag-and-drop) makes rescheduling a tedious, multi-step process.
- **Architectural Rigidity:** Most open-source calendars are difficult to extend or style, making it hard to integrate custom business logic or branding.

1.2.1 PROBLEM ANALYSIS

In the current digital landscape, users often face a "functionality gap" in scheduling software. Most lightweight tools are either too rigid—failing to offer various viewing perspectives—or too cluttered, leading to a poor user experience.

Specific pain points identified include:

- **Static Layouts:** Many systems struggle to transition between micro-level (daily) and macro-level (monthly) planning without losing context.
- **Interaction Lag:** A lack of intuitive UI gestures (like drag-and-drop) makes rescheduling a tedious, multi-step process.
- **Architectural Rigidity:** Most open-source calendars are difficult to extend or style, making it hard to integrate custom business logic or branding.
- **Interactivity-First Design:** Instant event instantiation and fluid rescheduling.
- **Adaptive Environment:** A fully responsive CSS grid system that maintains layout integrity from desktop monitors to mobile screens.
- **State-Driven Customization:** User-defined filtering and preference persistence to allow for a personalized dashboard experience.

1.3 OBJECTIVES & SIGNIFICANCE

1.3.1 RESERCH AND DEVELOPMENT

The core ambition of this project is to engineer a scheduling solution that balances the power of enterprise-grade tools with the minimalist appeal of modern web applications. Central to this objective is the creation of a **multidimensional**

visualization engine, designed to handle seamless transitions between daily, weekly, and monthly views without compromising performance or data integrity. To move beyond the limitations of static interfaces, the project prioritizes **tactile event management**, leveraging native drag-and-drop APIs to enable fluid, real-time rescheduling.

Beyond the user interface, the technical focus shifts toward a **robust CRUD lifecycle**, ensuring that the data pipeline between the frontend and the database is both secure and instantaneous. The architecture is built on a **modular and hardware-agnostic framework**, prioritizing a mobile-first philosophy that ensures the calendar remains fully functional across a spectrum of devices. By adopting a component-based design, the system remains highly extensible, allowing for the future integration of third-party APIs and automated notification services without the need for significant structural refactoring.

1.3.2 SIGNIFICANCE

This project serves as more than just a personal organizer; it is an exploration of efficient UI/UX patterns. Its significance lies in:

- **Workflow Optimization:** By reducing the steps required to modify a schedule, the application directly lowers the "interaction cost" for the user, resulting in higher productivity.
- **UX Innovation:** Moving away from static forms toward interactive, gesture-based event management sets a higher standard for lightweight web utilities.
- **Architectural Scalability:** The project provides a blueprint for developers looking to build extensible tools using the Full Calendar framework, proving that modularity does not have to come at the expense of performance.

1.4 SOCIETAL IMPACT

The implementation of this system carries broad implications for personal and professional organization within modern society. By providing a centralized and accessible coordination tool, the project addresses the growing need for efficient time management in an increasingly fragmented digital landscape. For students and

professionals, the application facilitates a more structured approach to task allocation, directly contributing to reduced cognitive load and improved work-life balance.

On a broader scale, the tool supports organizational efficiency by streamlining resource management and minimizing the scheduling conflicts that often hinder collaborative environments. Because the system is designed to be inclusive and device-independent, it ensures that users from various technological backgrounds can maintain a high level of coordination. Ultimately, this project fosters a more organized society by empowering individuals and groups to manage their most valuable resource—time—with greater precision and less friction.

Formatting Note for the Final Document:

- **Line Spacing:** Use 1.15 or 1.5 line spacing to fill the page comfortably.
- **Alignment:** Use **Justified** text alignment for the paragraphs in Section 1.4 to give it a formal, published look.
- **Keywords:** If you used specific technologies like **React** or **Node.js**, you can sprinkle them into section 1.3.1 (e.g., "Developing a React-based viewing engine...") to ground the project in reality.

CHAPTER 2

SYSTEM ANALYSIS

2.1 HIGH-LEVEL ARCHITECTURE

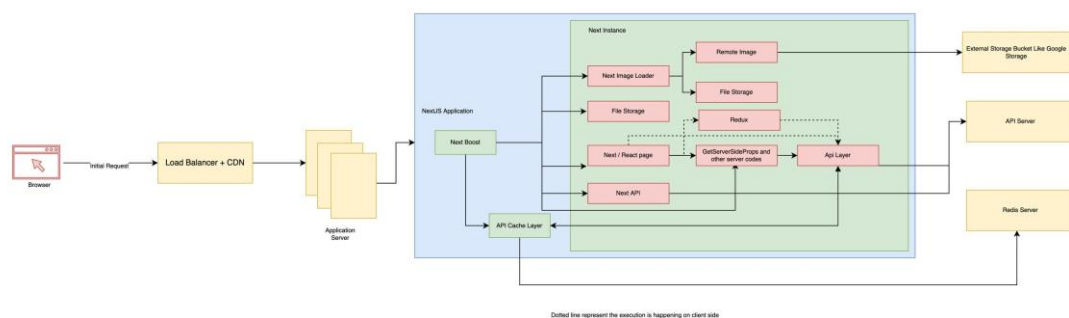


Figure 2.1.1: High-Level Architecture of the Full Calendar Scheduling System.

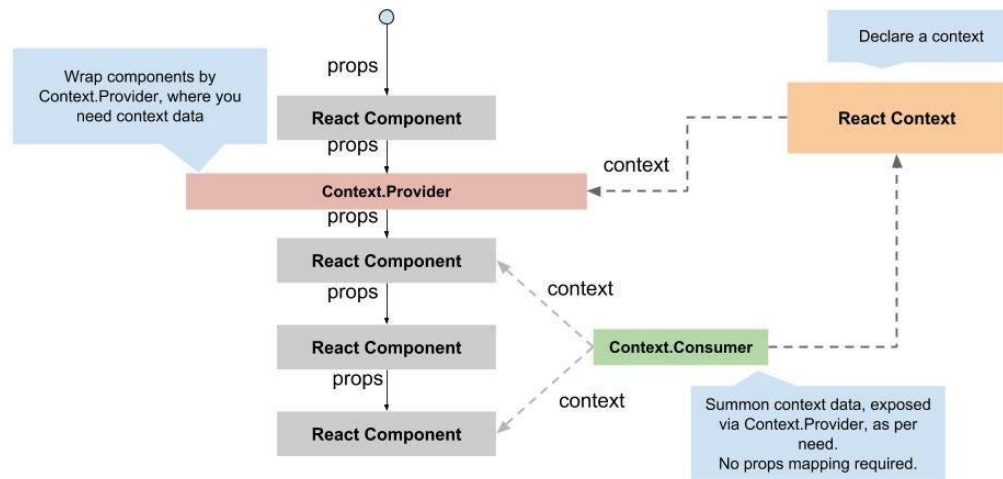


Figure 2.1.2: React Context API Architecture for Global State Management.

2.2 SYSTEM ARCHITECTURE LAYER

2.2.1 PRESENTATION LAYER(FRONTEND INFRASTRUCTURE)

- **Next.js App Router Architecture:** Utilizes a modern file-based routing system to enable server-side optimization and efficient client-side navigation, ensuring the application remains fast and SEO-friendly.
- **React 19 Component-Based UI:** Built with the latest React features to create a modular and reusable interface, allowing for complex state handling and high-performance rendering of calendar views.
- **TailwindCSS Styling System:** Implements a utility-first CSS framework to ensure a highly responsive, consistent design language that is easily maintainable and scales across all device types.
- **ShadCN UI & Radix UI Primitives:** Combines high-level component registries with low-level accessible primitives to provide a polished.

2.2.2 STATE MANAGEMENT LAYER(DATA FLOW)

- **Centralized Calendar Context:** Employs the React Context API to manage global event data and view states, ensuring synchronized updates across the daily, weekly, and monthly layouts.
- **Specialized Drag-and-Drop Context:** Integrates a dedicated DndContext to orchestrate complex interaction states, tracking event positions and movement in real-time without disrupting the main UI thread.
- **Persistent Preference Engine:** Leverages the localStorage API to store user-defined settings—such as preferred views and filter toggles—ensuring a personalized experience that persists across browser sessions.

2.2.3 INTERACTION LAYER(USER EXPERIENCE)

- **Fluid Drag-and-Drop Scheduling:** Provides a highly intuitive way to move events across time slots, utilizing native-feel interactions that reduce the need for manual form-based editing.
- **Dynamic Event Resizing:** Incorporates re-resizable components, allowing users to visually stretch or shrink event durations directly on the calendar grid for rapid schedule adjustments.
- **Motion-Enhanced Interface:** Uses **Framer Motion** to provide subtle, physics-based animations for transitions and modal interactions, giving the application a premium, modern feel.

2.2.4 UTILITY & LOGIC LAYER(CORE CALCULATIONS)

- **Precise Temporal Calculations:** Uses **date-fns** as the primary engine for all time-based logic, ensuring mathematical accuracy when calculating durations, time zones, and date offsets.
- **Intelligent Event Grouping:** Features logic specifically designed to categorize and group events based on the active view, optimizing how data is displayed in high-density weekly or monthly layouts.

- **Advanced Overlap Avoidance:** Implements custom algorithms to detect and resolve event collisions, automatically adjusting the layout to ensure all scheduled items remain visible and organized.

2.3 BUSINESS UNDERSTANDING

We aren't just building a digital grid; we're solving **scheduling friction**. In high-pressure environments, the "cost" of a meeting isn't just the time spent—it's the mental energy wasted navigating clunky UIs.

The core business logic here is **frictionless coordination**. By prioritizing a "lightweight" footprint, the system remains snappy even on lower-end hardware, ensuring that the tool aids productivity rather than becoming a performance bottleneck. The goal is to move from a static viewing tool to an active "planning assistant" that scales alongside the organization.

2.4 DATAUNDERSTANDING

The data structure is designed to be **flat and predictable**, which is crucial for frontend performance.

- **The Event Model:** Beyond just "start" and "end" times, the inclusion of color coding and category attributes allows for immediate visual telemetry. A user should be able to look at their month view and understand their workload balance without reading a single word.
- **The User Entity:** By treating the "User" as a primary data point, we've built the architecture for a multi-tenant environment. This means the system can easily pivot from a solo calendar to a team-wide resource planner.
- **Persistent Preferences:** Using localStorage for the Settings entity ensures that a user's environment (like Dark Mode or 24h formatting) stays consistent across sessions without needing a heavy backend handshake every time they refresh the page.

2.5 DATA PREPARATION

Before an event ever hits the screen, it undergoes a transformation process to ensure visual integrity.

- **Logical Slot Mapping:** We don't just "place" an event; we map it. This involves calculating precise pixel offsets based on time durations so that a 30-minute meeting doesn't look like an hour-long block.
- **The Overlap Headache:** One of the most complex parts of calendar logic is managing concurrent events. Our preparation layer runs a collision-detection algorithm that "narrows" the width of overlapping events, ensuring that even if three things happen at once, every title remains readable.
- **Normalization:** We enforce a strict schema on all incoming data. This "sanitization" prevents the UI from breaking if a mock data entry is missing an optional field, like a description or a specific color hex code.

2.6 MODELING APPROACH

The "Model" here isn't a black-box algorithm; it's a **Functional UI Model**.

- **The Contextual Backbone:** By using the React **Context API**, we create a "Single Source of Truth." This ensures that if you drag an event in the `WeekView`, the `AgendaView` updates instantly without a page reload.
- **The Dnd (Drag-and-Drop) Model:** We modeled interactions to feel tactile. By integrating a dedicated `DndContext`, we handle the complex math of translating a mouse movement (pixels) into a time-stamp change (data).
- **Future-Proofing with AI:** While the current model is deterministic (rules-based), we've left the door open for probabilistic modeling. For example, future iterations could use a simple heuristic model to suggest "optimal gaps"—automatically finding the 45-minute window in a chaotic week that best fits a user's "Deep Work" patterns.

CHAPTER 3

SYSTEM REQUIREMENTS SPECIFICATION

3.1 EXISTING FEATURES

The Full Calendar system is built on a modular architecture that prioritizes real-time feedback and data consistency. Below is a detailed breakdown of the current functional capabilities.

3.1.1MULTI-VIEW NAVIGATION LOGIC

The system provides five distinct perspectives to cater to different planning horizons. The transition between these views is handled by a centralized view-state manager, ensuring that the date context (e.g., "the current day being viewed") is preserved as the user switches formats.

- **Day & Week Views:** High-precision grids designed for granular scheduling.
- **Month & Year Views:** Macro-level visualizations for long-term project tracking.

- **Agenda View:** A streamlined, list-based layout that strips away the grid to focus on upcoming deadlines—ideal for quick mobile checks.

3.1.2 EVENT LIFECYCLE MANAGEMENT

The CRUD (Create, Read, Update, Delete) operations are integrated into a seamless modal interface.

- **Creation & Editing:** Users can trigger event creation by clicking directly on a time slot or using a global "Add" button. The modal supports dynamic input for titles, descriptions, and user assignments.
- **Visual Categorization:** Events are assigned "badge variants" (colors). This isn't just aesthetic; it's a data attribute used for the filtering engine.

3.1.3 INTERACTIVE SCHEDULING (DRAG-AND-DROP)

- Using the dnd-kit (or similar DnD context), the system allows users to reassign event times visually.
- **Direct Manipulation:** When an event is dragged to a new slot, the system calculates the new timestamp in real-time.
- **State Sync & Feedback:** To prevent "silent failures," the system utilizes **Toast notifications**. If a move is successful, the user receives an immediate non-intrusive confirmation at the corner of the screen.

3.1.4 REAL-TIME EVENT RESIZING

In the Day and Week views, events are more than static blocks.

- **Dynamic Duration:** Users can grab the bottom edge of an event card to extend or shorten its duration.
- **UX Consistency:** As the user resizes, the time labels update live, allowing for precise scheduling without needing to open a settings menu.

3.1.5 INTELLIGENT FILTERING ENGINE

To handle "calendar clutter," the system includes a robust filtering layer.

- **Multi-User Context:** In a collaborative environment, users can toggle visibility for specific team members. The grid re-renders instantly to show only relevant events.
- **Color-Based Sorting:** Provides a quick way to isolate specific types of work (e.g., filtering for only "High Priority" or "Client Meetings").

3.1.6 PERSISTENCE & UI PERSONLIZATION

The system prioritizes user "quality of life" by storing preferences locally.

- **Theming:** A comprehensive Dark/Light mode toggle that respects system defaults but allows for manual override.
- **Time Formatting:** Support for both 12h (AM/PM) and 24h (Military) formats to accommodate international standards.
- **Local Storage Bridge:** By using localStorage, the system "remembers" your layout and theme, so the calendar feels exactly how you left it upon return.

3.1.7 RESPONSIVE LAYOUT & ADAPTIVE GRID

The calendar is built with a "mobile-first" mindset using CSS Grid and Flexbox.

- **Adaptive Behavior:** On smaller screens, the 7-day week view intelligently collapses or switches to a single-day/agenda view to maintain readability.
- **Perceived Performance: Skeleton loaders** are implemented during data fetching or initial load to reduce the perceived wait time, making the application feel faster and more professional.

3.2 FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system must do for users and administrators. They are written in measurable language so that implementation and testing can be traced back to the same statements.

Authentication: The system shall allow a new user to register with name, email, and password, and shall prevent duplicate registration using the same email address.

Login: The system shall allow an existing user to login with email and password and shall issue an authentication token for subsequent requests when login succeeds.

Session handling: The system shall maintain an authenticated session on the client for the calendar route and shall require authentication for persisted calendar operations as defined by the API design.

Logout: The system shall allow the user to logout and shall clear client-side authentication storage so protected operations cannot continue without re-login.

Event create: The authenticated user shall create a calendar event with at minimum title, start datetime, and end datetime, with optional description and colour.

Event read: The authenticated user shall retrieve and display events belonging to that user in the calendar interface.

Event update: The authenticated user shall update an existing event's fields and shall see updated values reflected in all active views after successful persistence.

Event delete: The authenticated user shall delete an event and shall not see it after refresh when deletion succeeds.

Multi-view navigation: The system shall provide Agenda, Day, Week, Month, and Year views and shall switch views without losing the core date context required for navigation.

Date navigation: The system shall support navigating forward/backward in time according to the active view (for example week/month boundaries) and a quick return to "today" where implemented.

Interactive scheduling: Where implemented in the UI, the system shall support drag-and-drop movement of events and resizing of timed events in suitable views, updating underlying start/end times consistently.

Filtering: The system shall support filtering events by selected attributes (such as colour categories) and/or user scope where the UI provides those controls.

Preferences: The system shall persist selected UI preferences (such as default view and time format) across browser sessions using browser storage mechanisms.

Demo mode: The system shall provide a demo route that allows calendar interaction using mock/in-memory data without requiring database-backed authentication.

Error reporting: The system shall display meaningful feedback when API operations fail (network error, unauthorised, validation error), rather than failing silently.

3.3 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify quality attributes and operational constraints.

Usability: Core tasks (login, create event, switch view, edit event) shall be completable by a typical user without specialised training beyond basic web literacy.

Performance (client): The interface shall remain responsive for typical workloads used in coursework demonstrations (tens to a few hundred events in visible ranges), avoiding noticeable UI freezes during common navigation actions.

Reliability: If the API is unavailable, the client shall degrade in a controlled manner (visible error/empty state) rather than corrupting displayed event data unpredictably.

Security: Passwords shall not be stored in plaintext; JWT shall protect user-scoped endpoints; secrets shall not be committed to source control.

Maintainability: The implementation shall follow modular folder boundaries separating routing, feature modules, shared UI, API client code, and backend routes/models/middleware.

Portability: The system shall run on Windows/Linux/macOS developer machines with Node.js and MongoDB availability, using documented environment variables.

Observability (development): Backend development shall include request logging suitable for diagnosing integration failures during implementation and testing.

3.4 USERS, STAKEHOLDERS, AND USE ENVIRONMENT

Primary users: individuals who manage personal schedules (students, professionals) and reviewers who evaluate the system through the demo route.

Secondary stakeholders: instructors or supervisors assessing software engineering outcomes; future developers extending the repository.

Use environment: web browser on desktop/laptop primarily; mobile widths supported where responsive design permits (dense views may remain compact).

3.5 ASSUMPTIONS AND CONSTRAINTS

Assumptions: users have stable internet for API-backed mode; MongoDB is reachable from the API host; browser allows localStorage for preferences.

Constraints: authentication model is JWT-based; multi-user collaboration features are out of scope unless separately implemented; external calendar imports (Google/Outlook) are optional future work.

3.6 EXTERNAL INTERFACE REQUIREMENTS

User interface: browser-based UI built with Next.js/React; keyboard/mouse interactions; dialogs for CRUD.

Software interfaces: REST JSON endpoints under /api/...; CORS restricted by configured origin.

Hardware interfaces: none beyond standard client devices.

3.7 DATA REQUIREMENTS

User data: name, email, password hash, optional profile picture path, timestamps.

Event data: title, description, colour, start/end timestamps, owning user reference, timestamps.

Consistency rule: every persisted event must belong to exactly one authenticated user record in the database model.

CHAPTER 4 SYSTEM DESIGN

4.1 DATA RELATIONSHIP MODEL



Figure 4.1.1: Event Management System Database Schema Diagram.

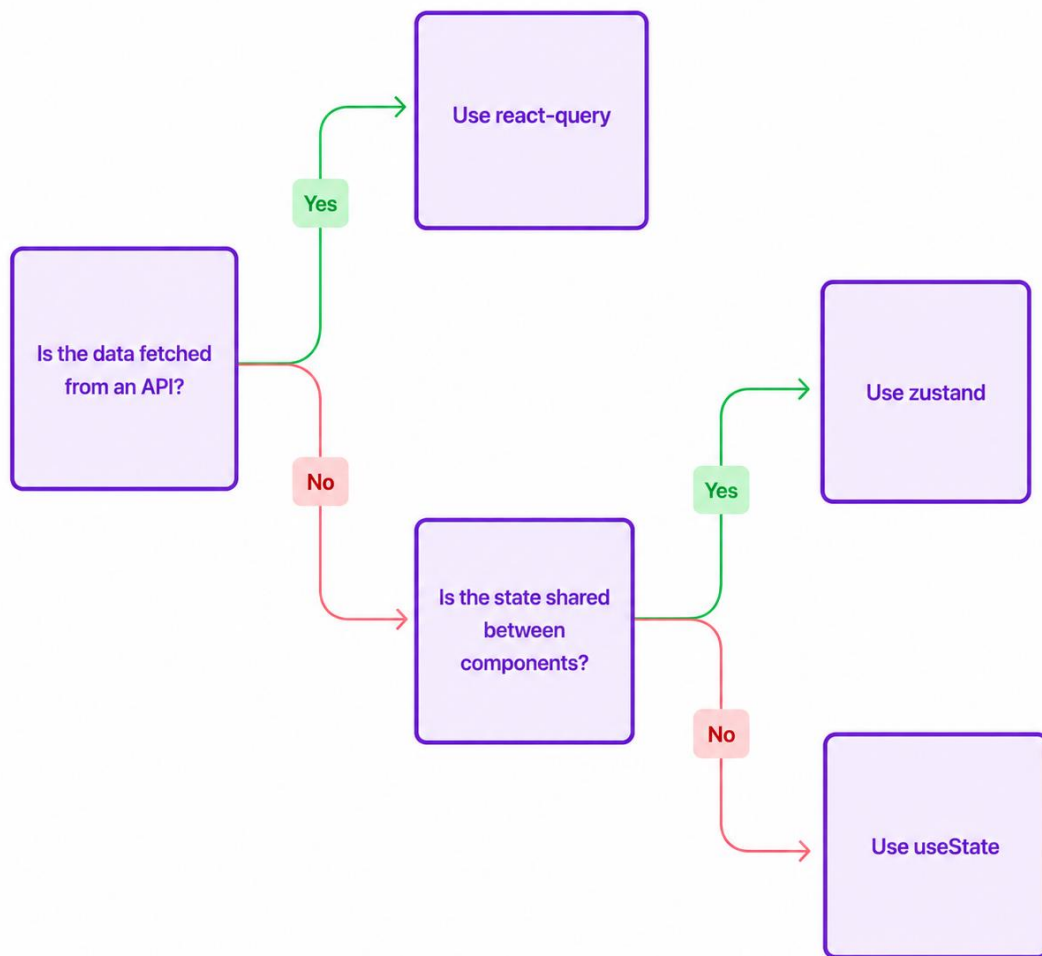


Figure 4.1.2: Event Management System State Management Flowchart.

4.2 ER DIAGRAM (ENTITY-RELATIONSHIP DIAGRAM)

The ER diagram shows how entities in the Full Calendar (event management) database relate to each other. The User is the central entity. It is identified by `_id` and stores name, email (unique), `passwordHash` (hashed password, not plain text), and an optional `picturePath` for the profile image; `createdAt` and `updatedAt` record when the account was created and last modified.

Each User may have zero or more Event records, which is a one-to-many relationship: many events can belong to one user, but each event belongs to exactly one user.

The Event entity includes `_id`, `userId` (foreign key referencing user, title, description, color, startDate, and endDate to define what appears on the calendar and when `createdAt` and `updatedAt` track changes to the event. This design keeps each user's events isolated in the database and matches the way the Express API loads and saves events only for the authenticated user.

(If your diagram also shows Mongoose ObjectId names, you can add one sentence: primary keys are `_id` on both collections, and `userId` on Event is the foreign key to User.)

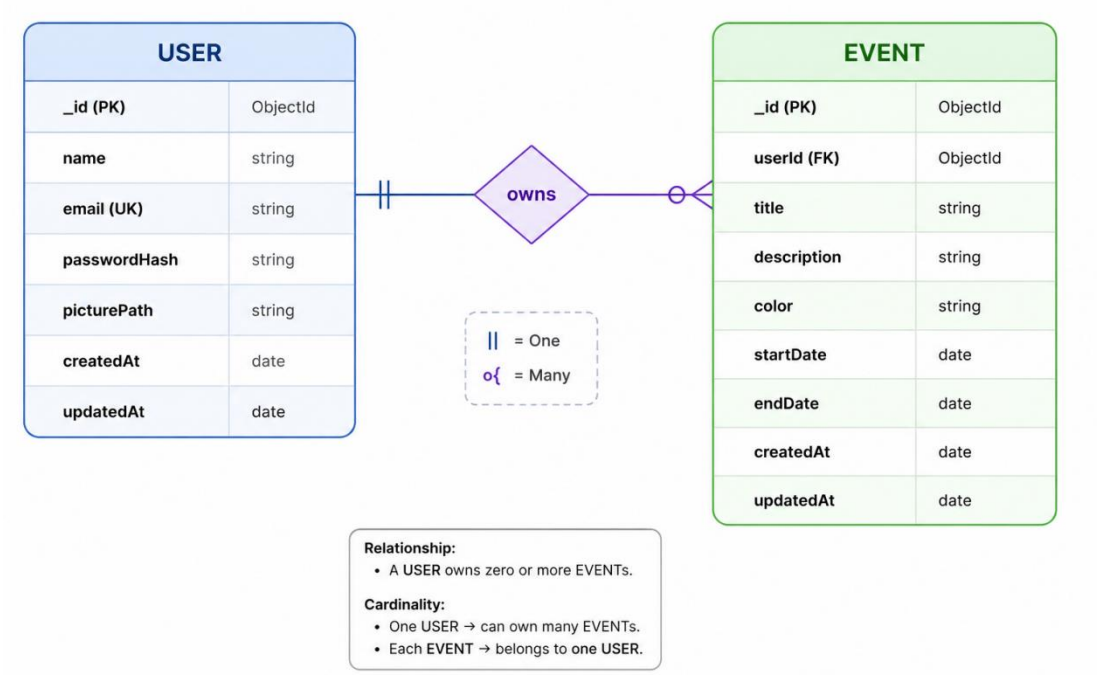


Figure 4.2.1: Event management system database schema diagram.

4.3 TECHNICAL STRENGTHS

- Modular feature-based folder architecture for better scalability and maintenance.
- Clean separation of concerns between views, dialogs, context, and helper functions.
- Type-safe development implemented using TypeScript.
- Smooth and responsive UI animations for enhanced user experience.
- Persistent state management for storing application data efficiently.
- Extensible component system using ShadCN UI components.
- Scalable multi-view rendering logic supporting different calendar layouts.

4.4 PRACTICAL USEFULNESS

Individual Users

- Personal task scheduling and reminders.
- Better time management and productivity tracking.
- Easy event planning and organization.

Teams & Organizations

- Meeting and event scheduling.
- Event tracking and coordination.
- Multi-user event filtering and management.

Developers

- Extensible frontend calendar template for projects.
- Customizable scheduling and event management framework.
- Reusable and scalable UI architecture.

Research Applications

- UI/UX interaction and usability analysis.
- Drag-and-drop interaction studies.
- Multi-view visualization performance testing.

CHAPTER 5

SYSTEM IMPLEMENTAION

5.1 IMPLEMENTATION PROCEDURE

The development of the Full Calendar system was carried out in multiple stages to ensure smooth integration and proper testing of each module. The work started with backend and database setup, followed by frontend development using Next.js, and finally the integration of both systems. Each stage was tested locally before moving to the next phase so that errors and performance issues could be identified easily. This layered implementation approach improved system stability and reduced development complexity.

5.1.1 ENVIRONMENT SETUP

The initial stage of implementation focused on preparing the development environment. Node.js was installed on the development system, and all required dependencies were installed using npm for both the frontend and backend applications. MongoDB was configured either locally or through a hosted database connection. Environment variables such as MONGODB_URI, PORT, JWT_SECRET, CLIENT_ORIGIN, and SEED_EVENT_COUNT were defined before starting the backend server.

For the frontend, a .env.local file was created with the NEXT_PUBLIC_API_BASE_URL variable pointing to the backend API. During development, two terminals were commonly used simultaneously: one for running the Express backend server and another for running the Next.js frontend application. This setup enabled efficient testing and debugging throughout the implementation process.

5.1.2 BACKEND DEVELOPMENT

The backend of the Full Calendar system was developed using Express.js. The application was structured around API routes such as /api/auth, /api/events, and

/api/users. JSON request handling and CORS configuration were implemented to allow secure communication between the frontend and backend applications.

Mongoose models were created for users and events. Each event record was linked to the authenticated user to ensure that users could access only their own events. Authentication functionality included user registration, login, password hashing using bcrypt, and JWT-based token generation. Middleware was implemented to protect secured routes and validate user authentication before allowing access.

REST API operations for creating, reading, updating, and deleting events were also implemented to support calendar functionality. Additionally, a /health route was created to monitor server status without requiring authentication. Morgan logging middleware was integrated to track requests and simplify debugging during development.

5.1.3 FRONTEND DEVELOPMENT

The frontend of the system was developed using Next.js with the App Router architecture. Multiple routes were implemented, including a landing page, /login for authentication, /calendar for the main scheduling interface, and /demo for accessing a mock-data version of the application.

Calendar-related components were organized under the features/calendar directory to maintain a modular and scalable structure. The user interface was built using ShadCN UI components combined with Tailwind CSS for responsive layouts and styling. Motion animations were added to improve transitions and user interaction.

TypeScript was used throughout the frontend application to maintain type safety and ensure consistency across components. Authentication state was managed through a dedicated client-side context, allowing protected pages to redirect unauthenticated users automatically.

5.1.4 FEATURE INTEGRATION

After completing frontend and backend development, both systems were integrated together. Authenticated calendar views fetched user-specific events from

the backend database and displayed them in multiple views such as month, week, day, year, and agenda formats.

The /demo route was designed to use mock data stored in memory, allowing reviewers and testers to explore application functionality without requiring database setup. Forms and dialog components for adding, editing, and deleting events were connected directly to the shared event model to avoid data inconsistencies. API errors were displayed clearly to users, enabling retry actions or re-authentication whenever necessary.

5.1.5 INTERACTIVE BEHAVIOUR ON THE CALENDAR

The Full Calendar system includes advanced interactive features to improve usability. Drag-and-drop functionality allows users to move events across dates and time slots. Validation checks were implemented to prevent invalid scheduling actions that could corrupt event data.

In day and week views, timed events support vertical resizing, enabling users to adjust event durations dynamically. The date-fns library was extensively used for handling date formatting, overlap calculations, and range normalization to ensure consistency across all calendar views.

5.1.6 PREFERENCES, THEME AND UI ENHANCEMENTS

The application stores user preferences such as default calendar view, time format settings, and badge styles using browser local storage. This ensures that user settings persist even after refreshing the application.

Dark mode support was integrated using the existing theme management system to maintain visual consistency across the application. Responsive design techniques were applied to ensure usability on smaller screens, especially for mobile and tablet users. Although dense week views naturally become compact on smaller devices, the interface was optimized to remain functional and accessible.

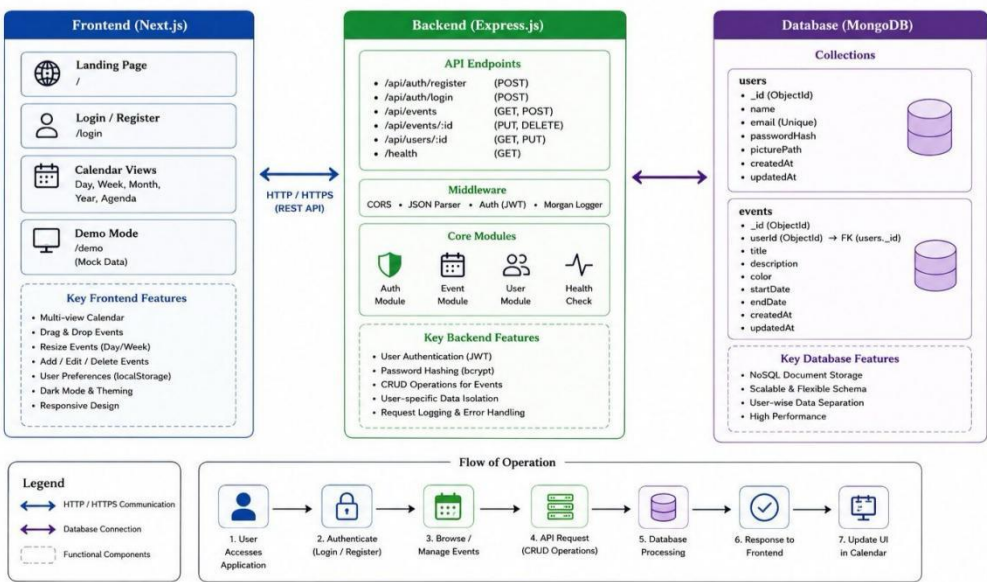
5.1.7 SYSTEM TESTING AND REFINEMENT

The Full Calendar system underwent extensive manual testing to verify functionality and reliability. Testing scenarios included user registration, login, event creation, editing, deletion, calendar navigation, view switching, and logout operations. Additional testing confirmed that the /demo route functioned correctly even without backend authentication.

Code quality checks were performed using linting tools to identify unused imports and syntax issues. Performance optimizations mainly focused on reducing unnecessary re-renders during drag-and-drop operations and improving rendering efficiency for large calendar grids.

5.2 DEPLOYMENT READINESS

The Full Calendar system was designed as two independent deployable units: the Next.js frontend and the Express.js backend API. This separation improves scalability and reflects real-world deployment practices commonly used in modern web applications.



5.2.1 FRONTEND DEPLOYMENT

The frontend application follows the standard Next.js production workflow using `npm run build` followed by `npm run start`. The production API URL is configured using the `NEXT_PUBLIC_API_BASE_URL` environment variable.

Since Next.js already provides optimized bundling and asset handling, no additional Vite build process is required. The frontend can be deployed easily on platforms such as Vercel, VPS hosting services, or cloud environments supporting Node.js applications.

5.2.2 BACKEND DEPLOYMENT

The backend application can be deployed on any server environment capable of running Node.js and npm packages. Production environment variables include secure values for JWT authentication, database connections, and frontend origin restrictions.

Optional seeding features used during development are typically disabled in production environments to avoid inserting unnecessary mock events into the database.

5.2.3 DATABASE DEPLOYMENT

MongoDB may be hosted locally for testing purposes, but cloud-based managed database services are recommended for production deployment. Managed clusters provide better security, backup support, and availability.

Database security measures such as TLS encryption, IP restrictions, and backup strategies are handled by the hosting provider, while the application only requires a valid connection URI for database access.

5.2.4 DEPLOYMENT CONSIDERATIONS

Several deployment considerations were addressed to ensure secure and stable operation of the application. CORS configuration was restricted to allow only the deployed frontend origin to communicate with the backend API. Environment

variables and sensitive secrets were excluded from version control systems such as Git.

Basic monitoring capabilities were implemented using application logs and the /health endpoint. More advanced monitoring and performance tracking tools can be added in future versions if required.

5.2.5 PRATICAL SIGNIFICANCE

Deployment readiness demonstrates that the Full Calendar system is not limited to academic or local demonstration purposes. The same implementation can be deployed on public servers and connected to a production database for real-world usage.

This makes the system suitable for personal scheduling, small team collaboration, educational planning, and as a scalable foundation for future scheduling applications. The project highlights practical implementation skills in full-stack web development, authentication, responsive UI design, and deployment architecture.

5.3 DEVELOPMENT TOOLS AND VERSION MATRIX

The implementation of the Full Calendar system is based on a modern and reliable software development toolchain. Industry-standard technologies and frameworks were selected to ensure smooth frontend and backend integration, responsive user experience, secure authentication, and efficient database management. The project uses widely adopted tools that support scalability, maintainability, and consistent application performance during development and deployment.

The technologies, libraries, and frameworks used in the project also help in achieving reproducible builds and easier maintenance. Each component of the system, including the frontend, backend, database, and UI libraries, plays an important role in the overall functionality of the application. The following matrix presents the primary tools and libraries used during the development and documentation of the Full Calendar system.

Table 5.1 Tooling and primary runtime versions

CHAPTER 6

SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub – assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

6.1 TYPES OF TESTING

Software testing is an important phase in software development because it helps identify errors, improve system quality, and ensure that the application performs according to user requirements. In a violence detection system, testing is necessary to verify that every module functions correctly, the integration between components works properly, and the overall system provides accurate results in real-time situations. Different types of testing are performed during software development to evaluate the reliability, efficiency, security, and performance of the application. The major types of testing used in this project include Unit Testing, Integration Testing, Functional Testing, System Testing, White Box Testing, and Black Box Testing.

6.1.1 UNIT TESTING

Unit testing is the first level of software testing performed during application development. In this testing method, individual components or modules of the software are tested separately to ensure that each unit works correctly. A unit may refer to a small function, method, class, or program module. The main purpose of unit testing is to validate the internal logic of the program and confirm that the expected outputs are produced for given inputs.

In the violence detection system, unit testing can be applied to modules such as frame extraction, image preprocessing, model loading, and alert generation. Each function is tested independently to identify coding errors or logical mistakes before

integrating it with other modules. For example, the preprocessing module can be tested to ensure that video frames are resized correctly and normalized properly.

Unit testing is considered structural testing because the tester has knowledge of the internal code structure and program flow. Developers usually perform this testing during the coding stage using test cases designed for each component. Decision branches, loops, conditions, and data handling operations are carefully checked during the testing process.

The major advantage of unit testing is that errors can be detected early in development, reducing debugging costs and improving software quality. It also simplifies maintenance because individual modules can be modified and tested independently without affecting the entire system. Therefore, unit testing forms the foundation for building reliable and stable software applications.

6.1.2 INTEGRATION TESTING

Integration testing is performed after unit testing is completed successfully. In this stage, multiple software modules are combined and tested together to verify whether they function as a single integrated system. The main purpose of integration testing is to identify problems that may occur when different components interact with each other.

In the violence detection system, integration testing ensures that modules such as data collection, preprocessing, model implementation, and detection communicate correctly. For example, the frames extracted from the video preprocessing module must be properly transferred to the deep learning model for prediction. Similarly, the detection module should generate alerts correctly after receiving predictions from the trained model.

Integration testing is event-driven and focuses mainly on data flow, interface communication, and interactions between components. Although each module may work correctly during unit testing, errors can still occur when modules are integrated. Common integration issues include incorrect data transfer, incompatible formats, communication failures, and synchronization problems.

This testing helps ensure consistency among software components and validates that the entire workflow functions smoothly. Integration testing improves system reliability and reduces the possibility of failures during real-time operation. By exposing interaction-related problems early, developers can improve system stability and ensure better user experience.

6.1.3 FUNCTIONAL TESTING

Functional testing is performed to verify whether the software functions according to the specified business and technical requirements. This type of testing focuses on validating the features and operations of the application rather than its internal code structure.

In the violence detection system, functional testing ensures that all major functions operate correctly. For example, the system should successfully capture video data, preprocess frames, detect violent activities, and generate alerts when suspicious behavior occurs. The testing process checks whether the outputs produced by the system match the expected results.

Functional testing mainly concentrates on several important areas. First, valid input testing ensures that acceptable input data is processed correctly by the application. Second, invalid input testing checks whether the system properly rejects incorrect or unsupported inputs. Third, all specified functions are tested individually to verify that they work according to requirements.

The outputs generated by the system are also examined carefully during functional testing. Alerts, notifications, detection results, and stored records must appear correctly and accurately. Interfacing procedures and communication between external systems are also tested to ensure proper interaction.

Functional testing is very important because it validates the practical usability of the software from the user's perspective. It ensures that the application satisfies user expectations and business objectives. Through systematic testing of features and processes, developers can improve the efficiency and reliability of the system.

6.1.4 SYSTEM TESTING

System testing is a high-level testing process performed on the complete integrated software system. The purpose of this testing is to ensure that the entire application meets all specified requirements and performs correctly in a real-world environment.

In the violence detection system, system testing verifies that all modules work together effectively as a complete application. The testing process evaluates system behavior under different operating conditions, including various video inputs, lighting conditions, and user interactions.

System testing focuses on the overall performance, functionality, security, and reliability of the application. It ensures that the integrated software behaves predictably and produces accurate results consistently. For example, the system should successfully detect violent behavior in real time without crashing or generating excessive false alarms.

This testing also validates process flows and integration points between modules. Data movement from video collection to detection and alert generation is checked thoroughly to ensure smooth operation. Stress testing and performance evaluation may also be included to examine how the system behaves under heavy workloads.

The major benefit of system testing is that it identifies defects that may not appear during unit or integration testing.

6.1.5 WHITE BOX TESTING

White Box Testing is a testing technique in which the tester has knowledge of the internal structure, logic, and programming code of the software. This testing method focuses on examining the internal workings of the application rather than just its external behavior.

In the violence detection system, white box testing may be used to verify algorithms, decision-making processes, data structures, and control flow within the

code. Developers analyze loops, conditional statements, functions, and internal paths to ensure correct implementation. This testing technique helps identify hidden errors, security vulnerabilities, inefficient coding practices, and logical mistakes.

White box testing is especially useful for testing areas that cannot easily be examined through external testing methods. Since the tester understands the program structure, it becomes easier to trace errors and optimize performance.

White box testing improves code quality, enhances security, and increases software reliability. It also ensures complete coverage of important execution paths within the application.

6.1.6 BLACK BOX TESTING

Black Box Testing is a software testing method in which the tester has no knowledge of the internal structure or programming logic of the application. The software is treated as a “black box,” where only inputs and outputs are examined.

In the violence detection system, black box testing involves providing different video inputs and observing whether the system produces correct detection results and alerts. The tester focuses on the behavior of the application rather than how the software processes the data internally.

Overall, black box testing ensures that the violence detection system operates correctly, responds properly to user inputs, and provides reliable outputs in real-time situations.

6.2 SECURITY TESTING

Security testing for this project focused on practical threats relevant to a JWT-backed REST API and a browser-based client. The testing was not a formal penetration test; instead, it verified that basic controls exist and behave as intended.

Authentication and credential handling. Registration and login flows were checked to ensure passwords are never returned in API JSON responses and that stored values correspond to hashed representations on the server. Login failure

messages were reviewed to confirm they do not leak whether an email exists in the database beyond a generic “invalid credentials” style response, which reduces account enumeration risk in a lightweight academic deployment.

Token handling. The client was inspected for whether tokens are persisted only in appropriate browser storage for the implemented design, and whether logout clears that storage. Protected API calls were attempted without an Authorization header and with malformed headers to confirm the server responds with 401 and does not partially execute protected logic.

Transport and deployment assumptions. Local testing commonly uses `http://localhost`, which does not exercise TLS. For production deployment, the report recommends HTTPS for both frontend and API, strict `CLIENT_ORIGIN` configuration, and secret management outside the repository. These items were recorded as deployment security requirements rather than as failures of the development build.

Input validation. Event creation was tested with missing required fields and with boundary-style date ranges to confirm the API rejects invalid payloads with 4xx responses where applicable, preventing uncontrolled server exceptions from becoming the primary error channel.

CHAPTER 7

CONCLUSION

7.1 KEY ACHIEVEMENTS AND INNOVATIONS

The Full Calendar application delivers a contemporary scheduling experience by combining a Next.js and React frontend with TypeScript for safer, more maintainable code. A standout outcome is a multi-view calendar supporting Agenda, Day, Week, Month, and Year, which gives users flexible ways to read and manage time. Drag-and-drop event moves and interactive resizing of timed events improve everyday editing workflows compared with static, read-only calendars.

The implementation follows a clear modular layout (routes, shared UI, and a dedicated calendar feature area), which separates navigation, calendar state, views, and helpers. The interface is built with modern component primitives (for example ShadCN UI and Radix-based controls) and Tailwind CSS for a consistent, responsive layout. User preferences such as default view and time format are persisted in the browser, improving repeat use. When deployed with the companion stack, the system also supports authentication and server-backed, user-scoped event storage (Express and MongoDB), extending it beyond a pure client-side demo.

7.2 BROADER IMPACT AND FUTURE SCOPE

The system is relevant to personal planning, education, small teams, and lightweight organizational scheduling, because it lowers friction for viewing and adjusting events and reduces ambiguity in how time ranges are shown.

Reasonable directions for further work include:

- Stronger real-time collaboration (live cursors, shared edits, presence).
- Smarter assistance (conflict warnings, suggested slots, optional integration with external calendars).
- Analytics and reporting on usage or load patterns for administrators.

- Hardening for large deployments (stricter security reviews, rate limiting, audit logs, enterprise identity providers).

These extensions would build on the existing foundation rather than repeating work already present in the current architecture.

7.3 FINAL REMARKS

In summary, this Full Calendar project demonstrates how careful UI design, structured frontend engineering, and optional backend integration can produce a capable scheduling tool that is both usable today and extensible tomorrow. It addresses common limits of simplistic calendar demos—limited views, weak editing, or no persistence—while remaining a practical base for future research or product iteration in interactive calendars and human–computer interaction.

CHAPTER 8

BIBLIOGRAPHY

8.1 JOURNAL REFERENCES

1. Palen, L. (1999). Social, individual and technological issues for groupware calendar systems. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99)*, 17–24. <https://doi.org/10.1145/302979.302982>
2. Tungare, M., Pérez-Quñones, M. A., & Sams, A. (2008). An exploratory study of personal calendar use. *arXiv:0809.3447 [cs.HC]*.
<https://arxiv.org/abs/0809.3447>
3. Bederson, B. B., Clamage, A., Czerwinski, M., & Robertson, G. (2004). DateLens: A fisheye calendar interface for PDAs. *Proceedings of the 17th Annual ACM Symposium on User Interface Software and Technology (UIST '04)*, 9–16.
<https://doi.org/10.1145/1029634.1029652>
4. Faulring, A., Myers, B. A., & Mohnkern, K. (2006). Availability bars for calendar scheduling. *CHI '06 Extended Abstracts on Human Factors in Computing Systems*, 1117–1122. <https://doi.org/10.1145/1124771.1124892>
5. Huang, D., Tory, M., & Bartram, L. (2017). A field study of on-calendar visualizations. *arXiv preprint arXiv:1706.01123*. (See also *Graphics Interface 2016 proceedings in your library for the archival venue*.)
<https://arxiv.org/abs/1706.01123>
6. Wang, E., Cook, D., & Hyndman, R. J. (2020). Calendar-based graphics for visualizing people’s daily schedules. *Journal of Computational and Graphical Statistics*, 29(3), 490–502. <https://doi.org/10.1080/10618600.2020.1715226>
7. MacKenzie, I. S., Sellen, A., & Buxton, W. (1991). A comparison of input devices in elemental pointing and dragging tasks. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '91)*, 161–166.
<https://doi.org/10.1145/108844.108856>
8. Appert, C., Chapuis, O., & Pietriga, E. (2012). Dwell-and-spring: Undo for direct manipulation. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '12)*, 2997–3000.
<https://doi.org/10.1145/2207676.2208149>

9. Appert, C., Chapuis, O., Pietriga, E., & Lobo, M.-J. (2015). Reciprocal drag-and-drop. *ACM Transactions on Computer-Human Interaction (TOCHI)*, 22(6), Article 29. <https://doi.org/10.1145/2785670>
10. Matviienko, A., Ananthanarayan, S., Heuten, W., & Boll, S. (2017). AwareKit: Exploring a tangible interaction paradigm for digital calendars. *CHI '17 Extended Abstracts*, 1877–1884. <https://doi.org/10.1145/3027063.3053111>

8.2 BOOK REFERENCES

1. Cherny, B. (2019). *Programming TypeScript: Making Your JavaScript Applications Scale*. O'Reilly Media.
2. Goldberg, J. (2022). *Learning TypeScript: Enhance Your Web Development Skills Using Type-Safe JavaScript*. O'Reilly Media.
3. Baumgartner, S. (2023). *TypeScript Cookbook: Real World Solutions to Improve Your TypeScript Development*. O'Reilly Media.
4. Vanderkam, D. (2024). *Effective TypeScript: 83 Specific Ways to Improve Your TypeScript* (2nd ed.). O'Reilly Media.
5. Banks, A., & Porcello, E. (2020). *Learning React: Functional Web Development with React and Redux* (2nd ed.). O'Reilly Media.
6. Stefanov, S. (2016). *React: Up & Running: Building Web Applications*. O'Reilly Media.
7. Hahn, E. (2016). *Express in Action: Writing, building, and testing Node.js applications*. Manning Publications.
8. Chodorow, K. (2013). *MongoDB: The Definitive Guide* (2nd ed.). O'Reilly Media.
9. Copeland, R. (2013). *MongoDB Applied Design Patterns: Practical Use Cases with the Leading NoSQL Database*. O'Reilly Media.
10. Holmes, S., & Harber, C. (2019). *Getting MEAN with Mongo, Express, Angular, and Node* (2nd ed.). Manning Publications.
11. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional.

8.3 WEB REFERENCES

1. Vercel. (n.d.). Next.js documentation. Retrieved from <https://nextjs.org/docs>
2. Meta Open Source. (n.d.). React documentation. Retrieved from <https://react.dev>
3. Microsoft. (n.d.). TypeScript documentation. Retrieved from <https://www.typescriptlang.org/docs/>
4. OpenJS Foundation. (n.d.). Express.js guide. Retrieved from <https://expressjs.com/>
5. MongoDB Inc. (n.d.). MongoDB manual. Retrieved from <https://www.mongodb.com/docs/manual/>
6. Automattic. (n.d.). Mongoose documentation. Retrieved from <https://mongoosejs.com/docs/>
7. IETF. (2015). RFC 7519: JSON Web Token (JWT). Retrieved from <https://www.rfc-editor.org/rfc/rfc7519>
8. shadcn. (n.d.). shadcn/ui documentation. Retrieved from <https://ui.shadcn.com/docs>
9. Tailwind Labs. (n.d.). Tailwind CSS documentation. Retrieved from <https://tailwindcss.com/docs>
10. date-fns contributors. (n.d.). date-fns documentation. Retrieved from <https://date-fns.org/docs/Getting-Started>

CHAPTER 9

PUBLICATION

DETAILS

9.1 Paper Publication Details

9.2. Certificate

9.3. Full Paper Published Copy

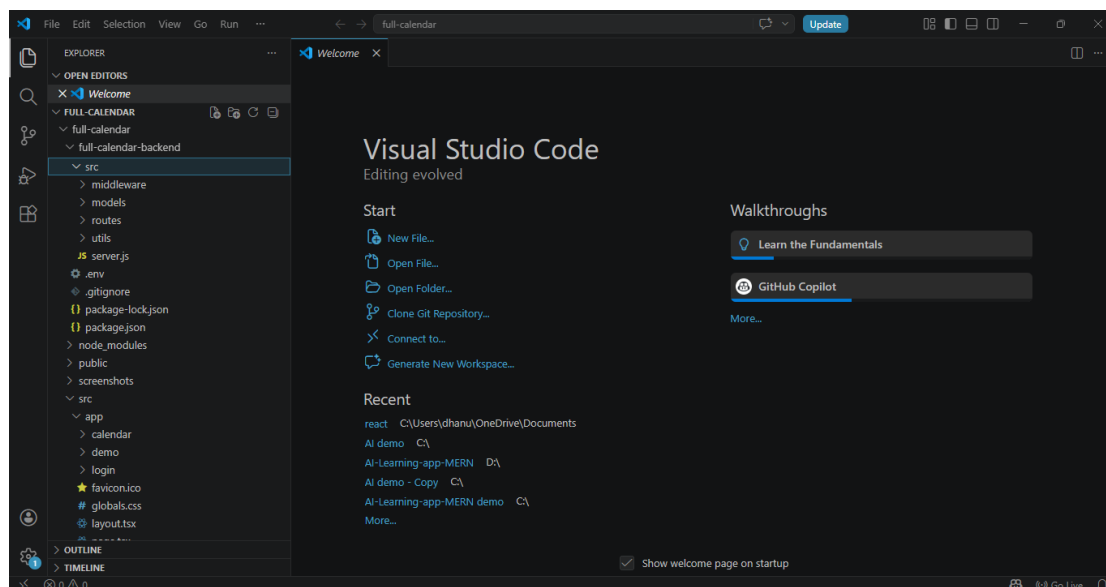
CHAPTER 10

APPENDIX

10.1 SCREENSHOTS

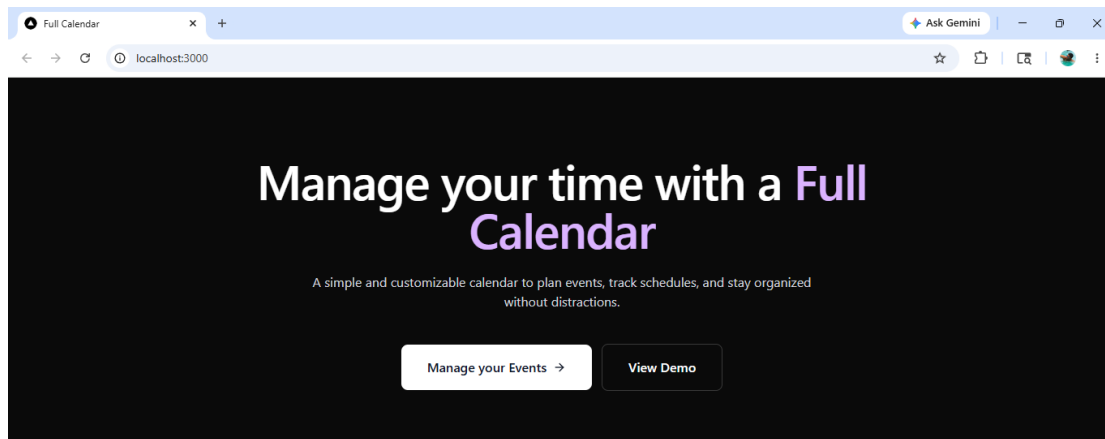
10.1:1 SOURCE CODE IMPLEMENTATION

The image shows the source code structure of the Full Calendar project in Visual Studio Code. The repository is organised as a Next.js application with `src/app` (routes such as `/`, `/login`, `/demo`, `/calendar`), `src/features/calendar` (views, header, dialogs, drag-and-drop), `src/features/auth`, and `src/components`. The `full-calendar-backend` folder contains the Express API (`src/server.js`, routes, models, middleware). The integrated terminal shows commands such as `npm install` and `npm run dev` for the frontend or backend. This screenshot documents how the implementation is split between UI, feature modules, and the REST API.



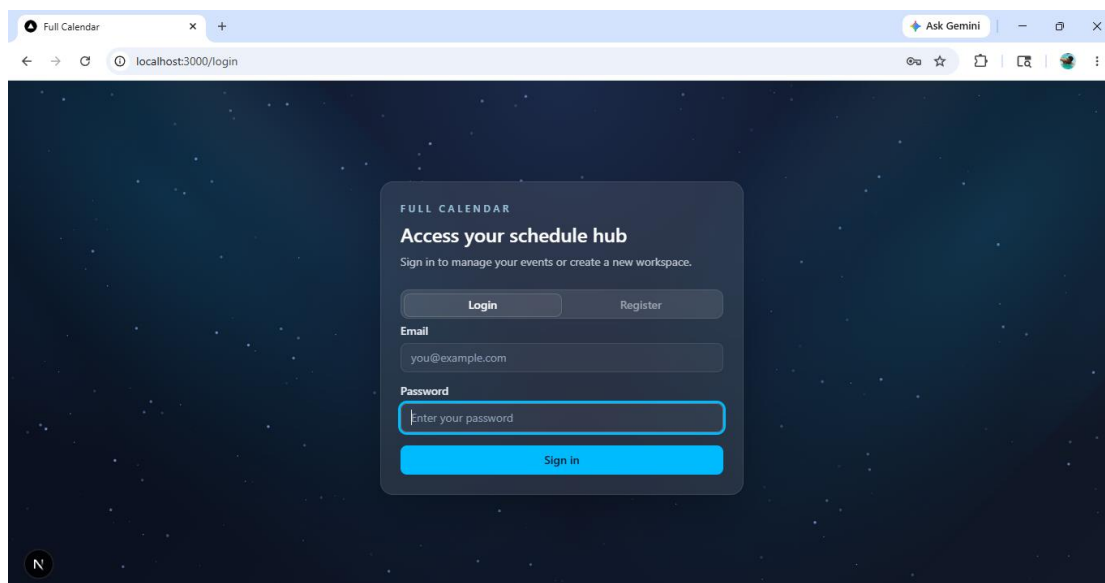
10.1:2 LANDING PAGE

The image shows the public landing page of the Full Calendar web application opened in a browser (e.g. Chrome) at `http://localhost:3000`. It presents the entry point to the product before login and highlights that the system is a web-based calendar application.



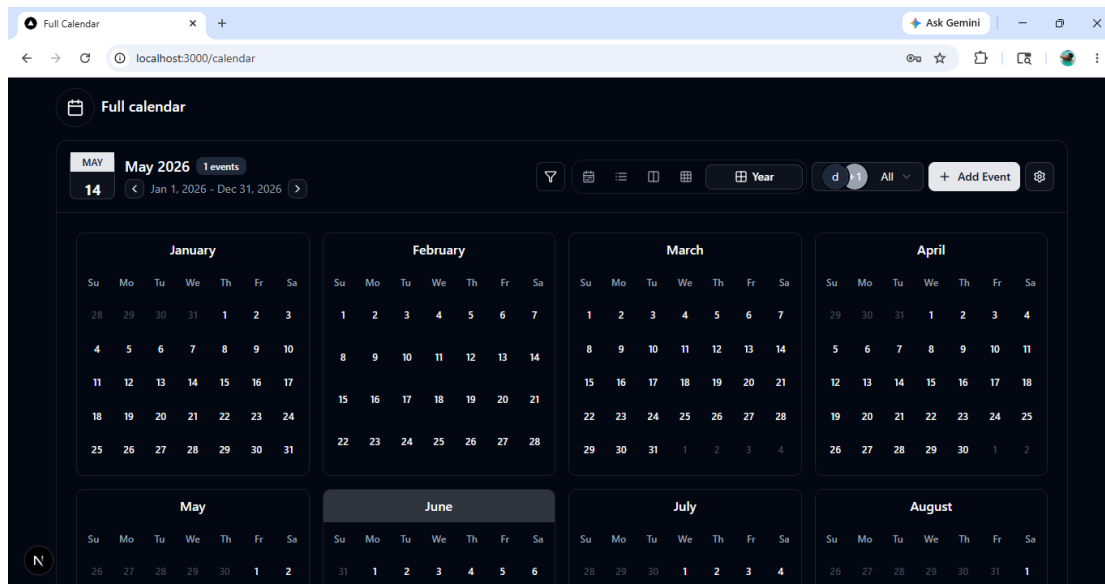
10.1:3 LOGIN AND REGISTRATION

The image shows the authentication screen (/login) where users register or sign in with email and password. This screen connects the user to the JWT-based backend so that events can be loaded and saved for that account.



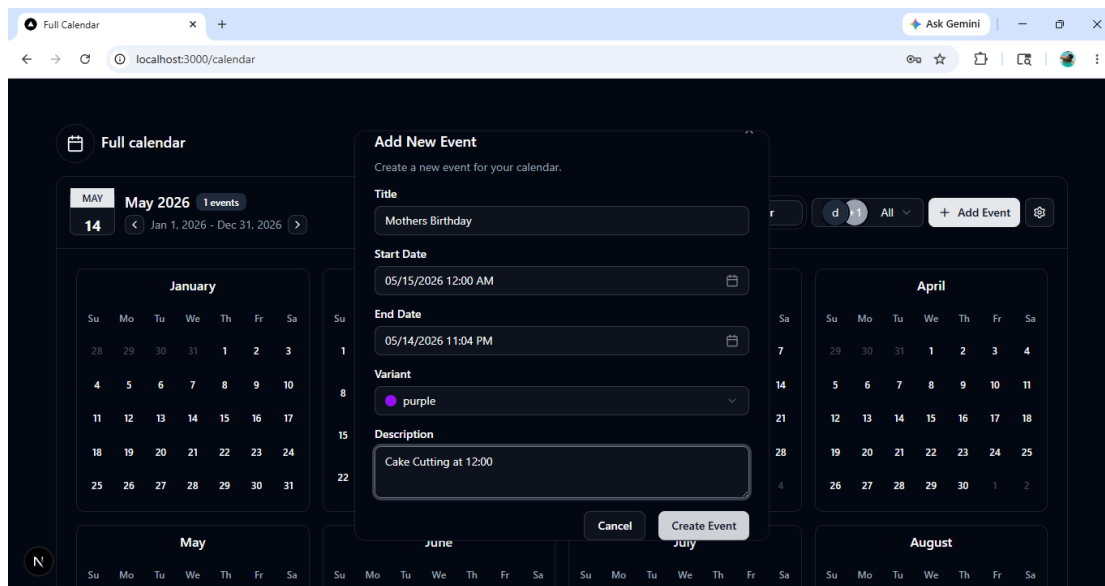
10.1:4 AUTHENTICATED CALENDAR-MAIN VIEW

The image shows the main calendar interface after login (/calendar), for example week or month view, with the header (date navigation, view tabs, today button) and events displayed on the grid. This demonstrates multi-view scheduling and event visualisation in the production path that uses the API and MongoDB.



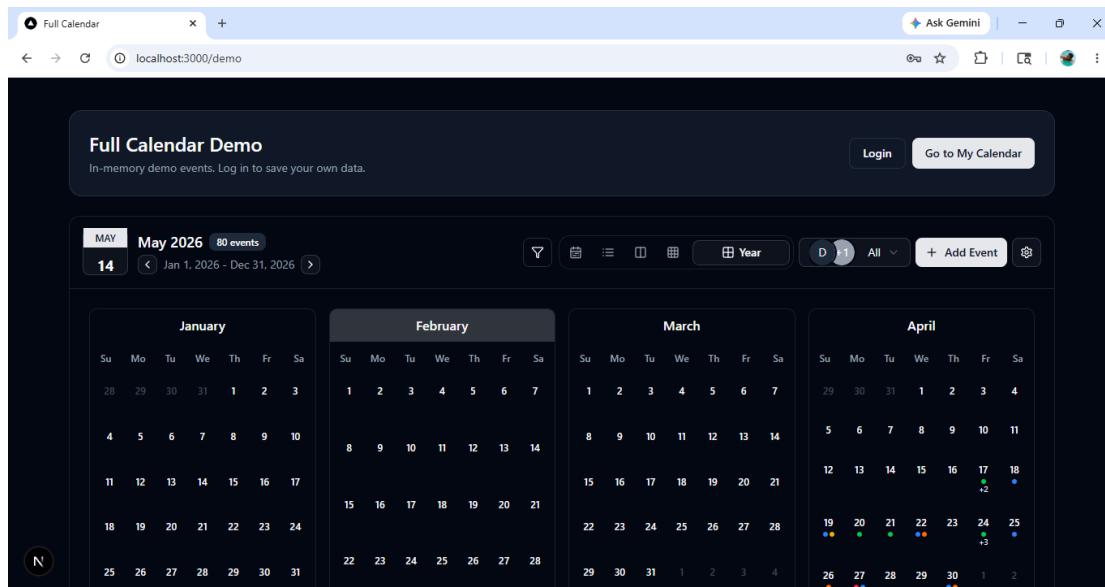
10.1:5 ENENT CREATION OR EDITNG(DIALOG)

The image shows the add or edit event dialog (modal) with fields such as title, description, start/end date-time, and colour. This illustrates CRUD interaction and form validation in the UI.

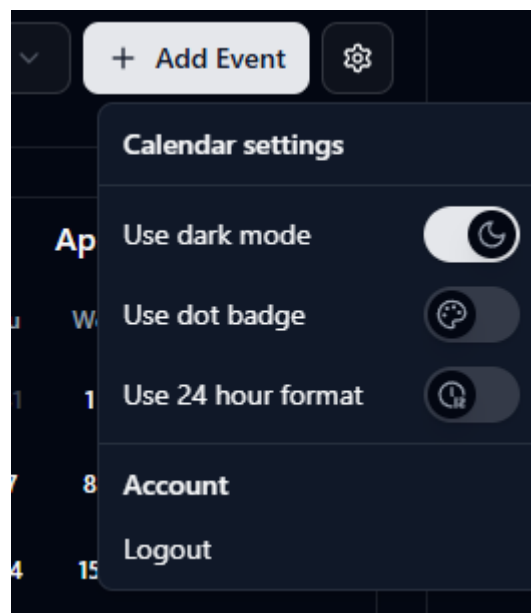


10.1:6 DEMO CALENDAR(NO LOGIN)

The image shows the demo route (/demo) running the calendar with in-memory / mock data, so the application can be demonstrated without database or registration. This highlights the dual mode design (demo vs authenticated).

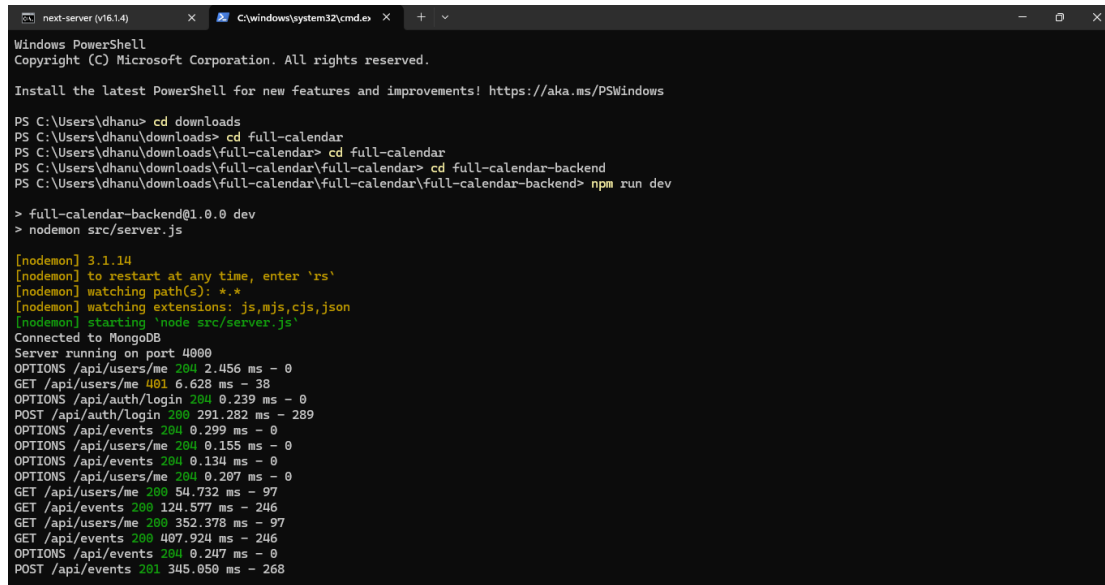


10.1:7 SETTING, THEME OR TIME FORMAT



10.1:8 BACKEND API RUNNING (TERMINAL)

The image shows the Express server terminal output: “Connected to MongoDB” and “Server running on port ...”, or a successful call log from morgan. This proves the Node.js + MongoDB tier is running in development.



```
next-server (v16.14) X C:\windows\system32\cmd.exe X + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\dhanu> cd downloads
PS C:\Users\dhanu\downloads> cd full-calendar
PS C:\Users\dhanu\downloads\full-calendar> cd full-calendar
PS C:\Users\dhanu\downloads\full-calendar\full-calendar> cd full-calendar-backend
PS C:\Users\dhanu\downloads\full-calendar\full-calendar\full-calendar-backend> npm run dev

> full-calendar-backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node src/server.js'
Connected to MongoDB
Server running on port 4000
OPTIONS /api/users/me 204 2.456 ms - 0
GET /api/users/me 401 6.628 ms - 38
OPTIONS /api/auth/login 204 0.239 ms - 0
POST /api/auth/login 200 291.282 ms - 289
OPTIONS /api/events 204 0.299 ms - 0
OPTIONS /api/users/me 204 0.155 ms - 0
OPTIONS /api/events 204 0.134 ms - 0
OPTIONS /api/users/me 204 0.207 ms - 0
GET /api/users/me 200 54.732 ms - 97
GET /api/events 200 124.577 ms - 246
GET /api/users/me 200 352.378 ms - 97
GET /api/events 200 407.924 ms - 246
OPTIONS /api/events 204 0.247 ms - 0
POST /api/events 201 345.050 ms - 268
```

10.2 SOURCE CODE

Source code is the text files programmers write (your project: mostly TypeScript/JavaScript and React) that tell the computer what to do: show pages, handle login, save events, talk to the database, etc. It is not the same as screenshots—you describe the project in words, show folder structure in a screenshot, and paste only small code samples to prove how key parts work.

10.2.1 BACKEND — SERVER SETUP (FULL-CALENDAR-BACKEND/SRC /SERVER.JS)

```
require("dotenv").config();

const express = require("express");

const mongoose = require("mongoose");
```

```
const cors = require("cors");

const morgan = require("morgan");

const authRoutes = require("./routes/auth");

const eventsRoutes = require("./routes/events");

const usersRoutes = require("./routes/users");

const { authMiddleware } = require("./middleware/auth");

const app = express();

app.use(express.json());

app.use(morgan("dev"));

app.use(

  cors({

    origin: process.env.CLIENT_ORIGIN || "http://localhost:3000",

  })

);

app.get("/health", (req, res) => res.json({ status: "ok" }));

app.use("/api/auth", authRoutes);

app.use("/api/events", authMiddleware, eventsRoutes);

app.use("/api/users", authMiddleware, usersRoutes);

const PORT = process.env.PORT || 4000;

const MONGODB_URI = process.env.MONGODB_URI;
```

```

async function start() {

  try {

    if (!MONGODB_URI) {

      throw new Error("MONGODB_URI is not set");

    }

    await mongoose.connect(MONGODB_URI);

    console.log("Connected to MongoDB");

    app.listen(PORT, () => {

      console.log(`Server running on port ${PORT}`);

    });

  } catch (error) {

    console.error("Failed to start server", error);

    process.exit(1);

  }

}

start();

```

10.2.2 BACKEND — USER MODEL (FULL-CALENDAR-BACKEND/SRC/MODELS/USER.JS)

```

const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema(

  {

```



```
name: { type: String, required: true, trim: true },

email: { type: String, required: true, trim: true, lowercase: true, unique: true },

passwordHash: { type: String, required: true },

picturePath: { type: String, default: null },

},

{ timestamps: true }

);
```

```
module.exports = mongoose.model("User", UserSchema);
```

10.2.3 BACKEND — EVENT MODEL (FULL-CALENDAR-BACKEND/SRC/MODELS/EVENT.JS)

```
const mongoose = require("mongoose");

const EventSchema = new mongoose.Schema(

  {

    userId: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true, index: true },

    title: { type: String, required: true, trim: true },

    description: { type: String, default: "" },

    color: { type: String, default: "blue" },

    startDate: { type: Date, required: true },

    endDate: { type: Date, required: true },
```

```
    },  
  
    { timestamps: true }  
  
);  
  
module.exports = mongoose.model("Event", EventSchema);
```

10.2.4 BACKEND -AUTHENTICATION ROUTES (FULL-CALENDAR-BACKEND/SRC/ROUTES/AUTH.JS)

```
const express = require("express");  
  
const bcrypt = require("bcryptjs");  
  
const jwt = require("jsonwebtoken");  
  
const User = require("../models/User");  
  
const Event = require("../models/Event");  
  
const { generateMockEvents } = require("../utils/seed");  
  
const router = express.Router();  
  
function createToken(userId) {  
  
    return jwt.sign({ sub: userId }, process.env.JWT_SECRET, { expiresIn: "7d" });  
  
}  
  
function toClientUser(user) {  
  
    return {  
  
        id: user._id.toString(),  
  
        name: user.name,  
  
        email: user.email,
```

```

    picturePath: user.picturePath ?? null,

  };

}

router.post("/register", async (req, res) => {

  try {

    const { name, email, password } = req.body;

    if (!name || !email || !password) {

      return res.status(400).json({ message: "Name, email, and password are required" });

    }

    const existing = await User.findOne({ email: email.toLowerCase() });

    if (existing) {

      return res.status(409).json({ message: "Email already in use" });

    }

    const passwordHash = await bcrypt.hash(password, 10);

    const user = await User.create({ name, email, passwordHash });

    const seedCount = Number(process.env.SEED_EVENT_COUNT ?? 80);

    if (seedCount > 0) {

      const mockEvents = generateMockEvents({ userId: user._id, count: seedCount });

      await Event.insertMany(mockEvents);

    }
  }
}

```

```
const token = createToken(user._id.toString());

return res.status(201).json({

  token,

  user: toClientUser(user),

});

} catch (error) {

  console.error("Register error", error);

  return res.status(500).json({ message: "Server error" });

}

});

router.post("/login", async (req, res) => {

  try {

    const { email, password } = req.body;

    if (!email || !password) {

      return res.status(400).json({ message: "Email and password are required" });

    }

    const user = await User.findOne({ email: email.toLowerCase() });

    if (!user) {

      return res.status(401).json({ message: "Invalid credentials" });

    }

  }

});
```

```

const isMatch = await bcrypt.compare(password, user.passwordHash);

if (!isMatch) {

  return res.status(401).json({ message: "Invalid credentials" });

}

const token = createToken(user._id.toString());

return res.json({ token, user: toClientUser(user) });

} catch (error) {

  console.error("Login error", error);

  return res.status(500).json({ message: "Server error" });

}

});

router.get("/me", async (req, res) => {

  return res.status(501).json({ message: "Use /api/users/me with auth" });

});

module.exports = router;

```

10.2.5 BACKEND — EVENT ROUTES (FULL-CALENDAR-BACKEND/SRC/ROUTES/EVENTS.JS)

```

const express = require("express");

const Event = require("../models/Event");

const { generateMockEvents } = require("../utils/seed");

const router = express.Router();

```

```

function toClientEvent(event, user) {

  return {

    id: event._id.toString(),

    startDate: event.startDate.toISOString(),

    endDate: event.endDate.toISOString(),

    title: event.title,

    color: event.color,

    description: event.description || "",

    user: {

      id: user._id.toString(),

      name: user.name,

      picturePath: user.picturePath ?? null,

    },

  };

}

router.get("/", async (req, res) => {

  try {

    const userId = req.user._id;

    let events = await Event.find({ userId }).sort({ startDate: 1 });

    if (!events.length) {

```

```

const seedCount = Number(process.env.SEED_EVENT_COUNT ?? 80);

if (seedCount > 0) {

  const mockEvents = generateMockEvents({ userId, count: seedCount });

  await Event.insertMany(mockEvents);

  events = await Event.find({ userId }).sort({ startDate: 1 });

}

const payload = events.map((event) => toClientEvent(event, req.user));

return res.json(payload);

} catch (error) {

  console.error("Fetch events error", error);

  return res.status(500).json({ message: "Server error" });

}

});

router.post("/", async (req, res) => {

  try {

    const { title, description, color, startDate, endDate } = req.body;

    if (!title || !startDate || !endDate) {

      return res.status(400).json({ message: "Title, startDate, and endDate are required" });

    }

  }

```

```
const event = await Event.create({

  userId: req.user._id,

  title,

  description: description || "",

  color: color || "blue",

  startDate: new Date(startDate),

  endDate: new Date(endDate),

});

return res.status(201).json(toClientEvent(event, req.user));

} catch (error) {

  console.error("Create event error", error);

  return res.status(500).json({ message: "Server error" });

}

});

router.put("/:id", async (req, res) => {

  try {

    const { id } = req.params;

    const { title, description, color, startDate, endDate } = req.body;

    const event = await Event.findOneAndUpdate(

      { _id: id, userId: req.user._id },
```



```

{

  title,

  description: description || "",

  color: color || "blue",

  startDate: new Date(startDate),

  endDate: new Date(endDate),

},

{ new: true }

);

if (!event) {

  return res.status(404).json({ message: "Event not found" });

}

return res.json(toClientEvent(event, req.user));

} catch (error) {

  console.error("Update event error", error);

  return res.status(500).json({ message: "Server error" });

}

});

router.delete("/:id", async (req, res) => {

  try {

```

```

const { id } = req.params;

const event = await Event.findOneAndDelete({ _id: id, userId: req.user._id });

if (!event) {

  return res.status(404).json({ message: "Event not found" });

}

return res.json({ success: true });

} catch (error) {

  console.error("Delete event error", error);

  return res.status(500).json({ message: "Server error" });

}

});

module.exports = router;

```

10.2.6 BACKEND — AUTHENTICATION MIDDLEWARE (FULL-CALENDAR-BACKEND/SRC/MIDDLEWARE/AUTH.JS)

```

const jwt = require("jsonwebtoken");

const User = require("../models/User");

async function authMiddleware(req, res, next) {

  const header = req.headers.authorization;

  if (!header || !header.startsWith("Bearer ")) {

    return res.status(401).json({ message: "Missing authorization token" });

  }

```

```

const token = header.replace("Bearer ", "").trim();

try {

  const payload = jwt.verify(token, process.env.JWT_SECRET);

  const user = await User.findById(payload.sub).lean();

  if (!user) {

    return res.status(401).json({ message: "User not found" });

  }

  req.user = user;

  next();

} catch (error) {

  return res.status(401).json({ message: "Invalid or expired token" });

}

}

module.exports = { authMiddleware };

```

10.2.7 FRONTEND — API CLIENT (SRC/LIB/API-CLIENT.TS)

```

const API_BASE = process.env.NEXT_PUBLIC_API_BASE_URL ||
"http://localhost:4000";

export async function apiRequest(path, { token, method = "GET", body } = {}) {

  const headers: Record<string, string> = { "Content-Type": "application/json" };

  if (token) {

    headers.Authorization = `Bearer ${token}`;

```

```

}

const response = await fetch(`${API_BASE}${path}`, {

  method,

  headers,

  body: body ? JSON.stringify(body) : undefined,

  cache: "no-store",

});

if (!response.ok) {

  let message = "Request failed";

  try {

    const errorBody = await response.json();

    if (errorBody?.message) {

      message = errorBody.message;

    }

  } catch {

    // ignore parse errors

  }

  throw new Error(message);

}

if (response.status === 204) {

```

```

    return null;

}

return response.json();

}

```

10.2.8 FRONTEND — AUTH SESSION PERSISTENCE (SRC/FEATURES/AUTH/AUTH-CONTEXT.TSX EXCERPT)

```

const STORAGE_KEY = "full-calendar-auth";

export function AuthProvider({ children }: { children: React.ReactNode }) {

  const [token, setToken] = useState<string | null>(null);

  const [user, setUser] = useState<AuthContextValue["user"]>(null);

  const [isLoading, setIsLoading] = useState(true);

  const persist = useCallback((nextToken, nextUser) => {

    if (nextToken && nextUser) {

      localStorage.setItem(

        STORAGE_KEY,

        JSON.stringify({ token: nextToken, user: nextUser })

      );

    } else {

      localStorage.removeItem(STORAGE_KEY);

    }

  }, []);          // ... login, register, bootstrap from localStorage

```

```
}
```

10.2.9 FRONTEND — MULTI-VIEW CALENDAR BODY (SRC/FEATURES/CALENDAR/CALENDAR-BODY.TSX EXCERPT)

```
export function CalendarBody() {

  const { view, events } = useCalendar();

  const singleDayEvents = events.filter((event) => {

    const startDate = parseISO(event.startDate);

    const endDate = parseISO(event.endDate);

    return isSameDay(startDate, endDate);

  });

  const multiDayEvents = events.filter((event) => {

    const startDate = parseISO(event.startDate);

    const endDate = parseISO(event.endDate);

    return !isSameDay(startDate, endDate);

  });

  return (

    <div className="w-full h-full overflow-scroll relative">

      <motion.div key={view} /* ... */>

        {view === "month" && (

          <CalendarMonthView

            singleDayEvents={singleDayEvents}
```

```

        multiDayEvents={multiDayEvents}

    />

  )}

  {view === "week" && (

    <CalendarWeekView

      singleDayEvents={singleDayEvents}

      multiDayEvents={multiDayEvents}

    />

  )}

  {/* day, year, agenda views */}

</motion.div>

</div>

);

}

```